PROJECT REPORT

(CSA2001 Fundamentals in AI & ML)

AI Disease Prediction System

Ankesh Raj
Registration No.- 25BAI11393

Slot-B11+B12+B13
Class ID- BL2025260500824
Winter Semester: 2025-2026

Introduction

Project Title-
AI-Powered Disease Prediction System Using Machine Learning

The main goal of this project was to build a command-line tool that can predict diseases based on the symptoms a user enters. I used a Decision Tree Classifier for this. The idea was simple — you type in what you're feeling, and the system tells you what disease it could be, along with a confidence score, a short description of that disease, and some precautions you should take.
A lot of people, especially in places where healthcare isn't easily accessible, don't really know what their symptoms could mean. They might Google stuff and end up panicking over nothing, or worse, they ignore something serious. I wanted to build something that could take in symptoms and give a reasonable prediction using actual data, not just random internet articles. It is obviously not a replacement for a doctor, but it can give people a heads-up about what might be going on. I also added fuzzy matching so even if someone misspells a symptom, the system can still figure out what they meant.

Objective-
- Train a machine learning model on a medical symptom dataset
- Build an interactive prediction tool that handles messy user input
- Show useful info like disease descriptions, precautions, and severity ratings
- Generate visual charts to evaluate how well the model performs

Review-

I investigated how machine learning is being used in healthcare, and Decision Trees kept coming up as a solid choice for this kind of classification problem. Here is why I went with it:

- **Interpretability** — you can trace how the model makes decisions, which is important when you are dealing with health stuff
- **Speed** — it trains fast and gives predictions almost instantly
- **Works well with binary data** — since our features are just 0s and 1s (symptom present or not), Decision Trees handle this well

I also used the `thefuzz` library for fuzzy string matching. Basically, if someone types "headache" instead of "headache", the system still picks it up. This makes the tool way more practical because nobody types perfectly every time.

System Architecture-

```
┌──────────────────────────────────────────────────────────────────────┐
│                          AI DISEASE PREDICTION SYSTEM                 │
├──────────────────────────────┬───────────────────────────────────────┤
│      TRAINING MODULE         │          PREDICTION MODULE            │
│       (train.py)             │           (predict.py)                │
├──────────────────────────────┼───────────────────────────────────────┤
│                              │                                       │
│   ┌──────────────────┐       │    ┌──────────────────────────┐       │
│   │  Dataset (ZIP)   │       │    │  User Symptom Input      │       │
│   └──────────────────┘       │    └──────────────────────────┘       │
│           │                  │                │                      │
│           ▼                  │                ▼                      │
│   ┌──────────────────┐       │    ┌──────────────────────────┐       │
│   │  Extract & Load  │       │    │  Fuzzy Matching Engine   │       │
│   │  CSV Files       │       │    │  (thefuzz library)       │       │
│   └──────────────────┘       │    └──────────────────────────┘       │
│           │                  │                │                      │
│           ▼                  │                ▼                      │
│   ┌──────────────────┐       │    ┌──────────────────────────┐       │
│   │  Train Decision  │       │    │  Build Feature Vector    │       │
│   │  Tree Classifier │       │    │  + Severity Weights      │       │
│   └──────────────────┘       │    └──────────────────────────┘       │
│           │                  │                │                      │
│           ▼                  │                ▼                      │
│   ┌──────────────────┐       │    ┌──────────────────────────┐       │
│   │  Evaluate &      │──────────────►│  Model Prediction        │       │
│   │  Save Model      │       │    │  (predict_proba)         │       │
│   └──────────────────┘       │    └──────────────────────────┘       │
│           │                  │                │                      │
│           ▼                  │                ▼                      │
│   ┌──────────────────┐       │    ┌──────────────────────────┐       │
│   │  Generate Graphs │       │    │  Display Results:        │       │
│   │  confusion_matrix│       │    │  • Top-3 Predictions     │       │
│   │  classification  │       │    │  • Confidence Bars       │       │
│   │  _report         │       │    │  • Description           │       │
│   └──────────────────┘       │    │  • Precautions           │       │
│                              │    │  • Severity Rating       │       │
│                              │    └──────────────────────────┘       │
└──────────────────────────────┴───────────────────────────────────────┘
```

Technology Stack-

Component	Technology
Programming Language	Python 3.10+
ML Framework	scikit-learn
ML Algorithm	Decision Tree Classifier
Data Processing	Pandas, NumPy
Fuzzy Matching	thefuzz (fuzzywuzzy)
Visualization	Matplotlib
Data Format	CSV (inside ZIP archive)
Model Serialization	Pickle (.pkl)

Dataset Description-

Source-

I got the dataset from **Kaggle** it is the Disease Symptom Prediction Dataset, which comes as a `archive.zip` file.

Dataset Files-

File	Records	Description
`Training.csv`	306 rows	Training data with 134 symptom columns
`Testing.csv`	42 rows	Test data with the same structure
`disease_description.csv`	42 entries	Plain-English description
`disease_precaution.csv`	42 entries	Up to 4 precautionary measures
`symptom_severity.csv`	133 entries	Severity score (1-7) for each symptom

Key Statistics-

Metric	Value
Total disease classes	42
Total symptom features	134
Training samples	306
Testing samples	42
Feature type	Binary (0 = absent, 1 = present)
Target variable	`prognosis` (disease name)

Sample Diseases-

The dataset covers 42 diseases including AIDS, Acne, Allergy, Arthritis, Bronchial Asthma, Chicken Pox, Common Cold, Covid, Dengue, Diabetes, Fungal Infection, GERD, Heart Attack, Hepatitis (A-E), Hypertension, Jaundice, Malaria, Migraine, Pneumonia, Tuberculosis, Typhoid, and more.

Implementation Details

Training Pipeline (`train.py`)-

This script handles all the heavy lifting — from loading the data to training the model and saving everything.
Here is how it works:

1. It auto-detects the ZIP file in the project folder
2. Extracts the CSV files and finds `Training.csv` and `Testing.csv` (if it can't find them, it does a random 80/20 split as a fallback)
3. Fills in any missing values with 0
4. Trains a `DecisionTreeClassifier(random_state=42)` on 306 samples with 134 features
5. Tests it on 42 samples and gets a **97.62% accuracy**
6. Generates a confusion matrix heatmap and a bar chart showing per-disease classification metrics
7. Saves the model and all the extra data (descriptions, precautions, severity scores) as `.pkl` files
8. Cleans up any temporary files it created

Prediction Pipeline (`predict.py`)-

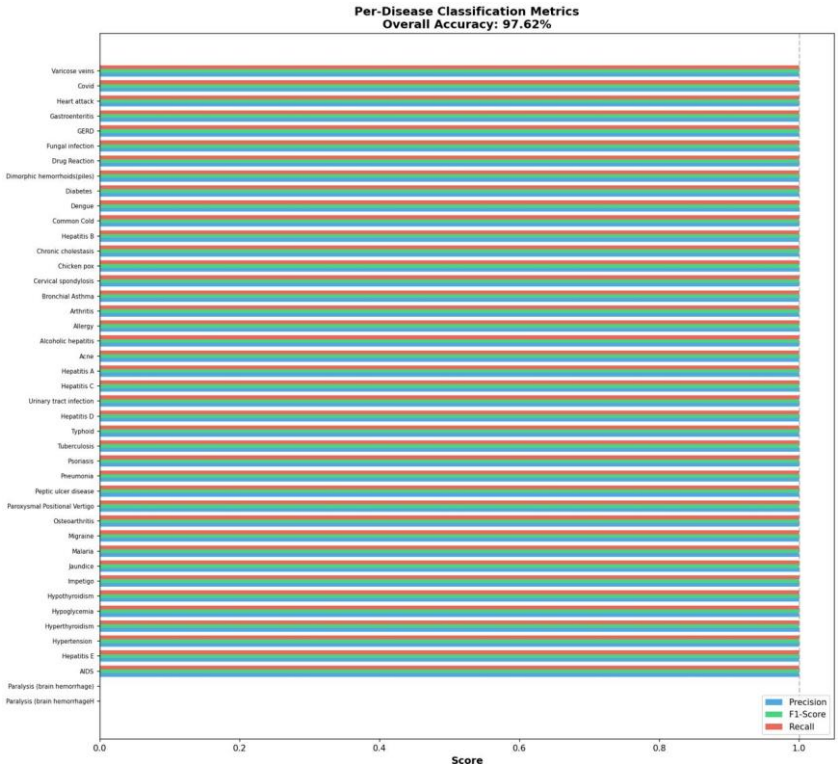
This is the part the user interacts with. Here is the flow:

```
User types symptoms (comma separated)
            |
            v
  Fuzzy Match via thefuzz (token_sort_ratio)
            |
            +-- Score >= 90 -> Auto-correct silently
            +-- Score 70-89 -> Auto-correct with notice
            +-- Score 50-69 -> Show top-3 options, user picks
            +-- Score < 50  -> Skip symptom
            |
            v
  Build 134-element feature vector
  Apply severity weights (1-7 scale)
            |
            v
  model.predict_proba(input_vector)
            |
            v
  Display:
    - Top-3 diseases with confidence % bars
    - Matched symptoms with severity table
    - Disease description (rank #1)
    - Precautions (rank #1)
    - Overall severity rating (Low/Medium/High)
```

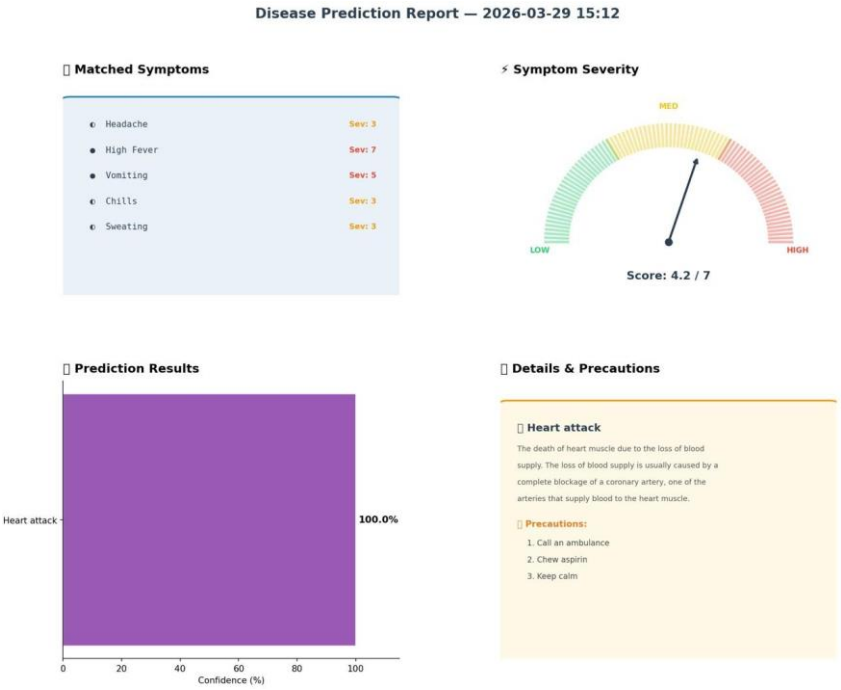
Algorithm-

1. It loads the trained model and all the extra data from the `.pkl` files
2. Shows all 134 available symptoms so you know what to type
3. You enter your symptoms separated by commas
4. Each symptom goes through a 4-tier fuzzy matching system (so typos are handled)
5. It builds a weighted feature vector using the severity scores
6. Runs `predict_proba()` to get probability-based rankings
7. Shows the top-3 predictions with confidence bars, a description, and precautions

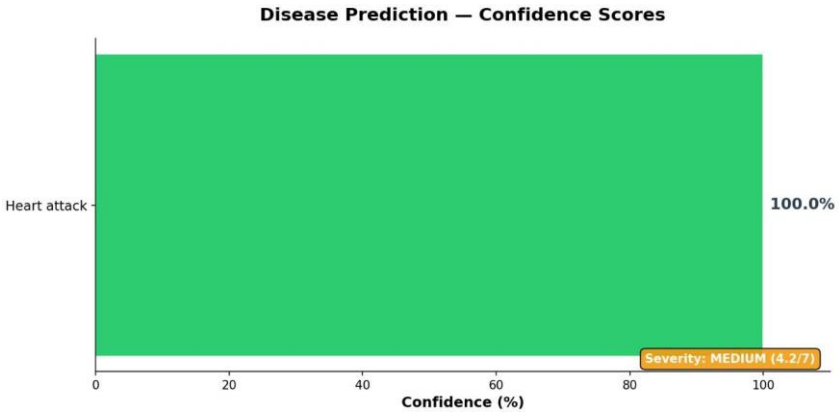
Results & Model Performance-



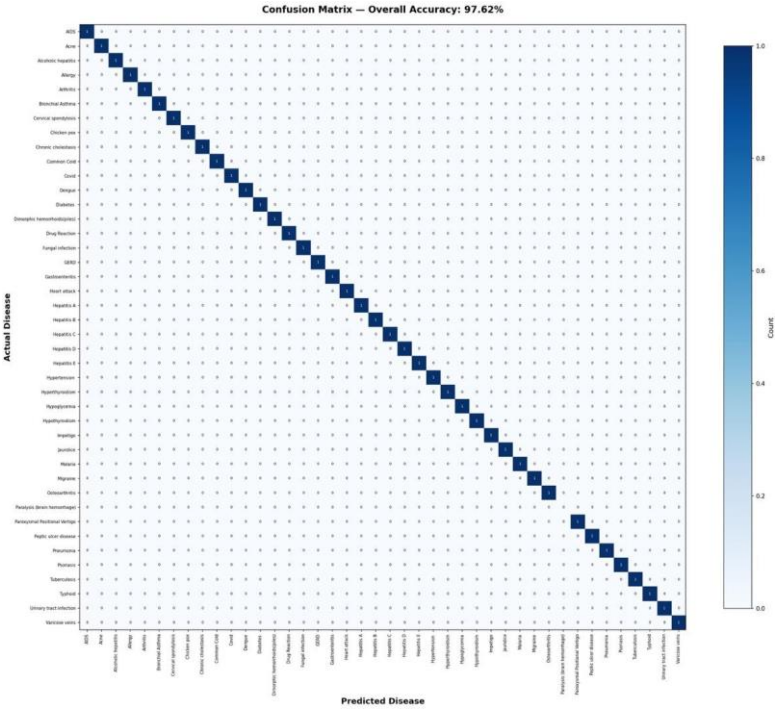
Model Performance Result 1



Model Performance Result 2



Model Performance Result 3



Model Performance Result 4

<u>Overall Metrics-</u>

Metric	Value
Test Accuracy	**97.62%**
Weighted Precision	0.98
Weighted Recall	0.98
Weighted F1-Score	0.98
Macro Precision	0.95
Macro Recall	0.95
Macro F1-Score	0.95

<u>Per-Disease Performance-</u>

Out of 42 diseases:

- **41 diseases** got **perfect scores** — 100% precision, recall, and F1
- **1 disease** (`Paralysis (brain hemorrhage)`) scored 0% — this happened because of a typo in the dataset where the label was `brain hemorrhage` instead of `brain hemorrhage`. So it is a data issue, not a model issue.

Full Classification Report

	precision	recall	f1-score	support
AIDS	1.00	1.00	1.00	1
Acne	1.00	1.00	1.00	1
Alcoholic hepatitis	1.00	1.00	1.00	1
Allergy	1.00	1.00	1.00	1
Arthritis	1.00	1.00	1.00	1
Bronchial Asthma	1.00	1.00	1.00	1
Cervical spondylosis	1.00	1.00	1.00	1
Chicken pox	1.00	1.00	1.00	1
Chronic cholestasis	1.00	1.00	1.00	1
Common Cold	1.00	1.00	1.00	1
Covid	1.00	1.00	1.00	1
Dengue	1.00	1.00	1.00	1
Diabetes	1.00	1.00	1.00	1
Dimorphic hemorrhoids(piles)	1.00	1.00	1.00	1
Drug Reaction	1.00	1.00	1.00	1
Fungal infection	1.00	1.00	1.00	1
GERD	1.00	1.00	1.00	1
Gastroenteritis	1.00	1.00	1.00	1
Heart attack	1.00	1.00	1.00	1
Hepatitis A	1.00	1.00	1.00	1
Hepatitis B	1.00	1.00	1.00	1
Hepatitis C	1.00	1.00	1.00	1
Hepatitis D	1.00	1.00	1.00	1
Hepatitis E	1.00	1.00	1.00	1
Hypertension	1.00	1.00	1.00	1
Hyperthyroidism	1.00	1.00	1.00	1
Hypoglycemia	1.00	1.00	1.00	1
Hypothyroidism	1.00	1.00	1.00	1
Impetigo	1.00	1.00	1.00	1
Jaundice	1.00	1.00	1.00	1
Malaria	1.00	1.00	1.00	1
Migraine	1.00	1.00	1.00	1
Osteoarthritis	1.00	1.00	1.00	1
Paralysis (brain hemorrhage)	0.00	0.00	0.00	1
Paroxysmal Positional Vertigo	1.00	1.00	1.00	1
Peptic ulcer disease	1.00	1.00	1.00	1
Pneumonia	1.00	1.00	1.00	1
Psoriasis	1.00	1.00	1.00	1
Tuberculosis	1.00	1.00	1.00	1
Typhoid	1.00	1.00	1.00	1

```
Urinary tract infection       1.00      1.00      1.00         1
          Varicose veins       1.00      1.00      1.00         1

                accuracy                           0.98        42
               macro avg       0.95      0.95      0.95        42
            weighted avg       0.98      0.98      0.98        42
```

Analysis-

The 97.62% accuracy makes sense when you think about it — each disease in this dataset has a unique set of symptoms that does not overlap much with other diseases. Since all the features are binary (either you have the symptom or you do not), Decision Trees can draw clear decision boundaries easily. It is kind of the perfect scenario for this algorithm.

Generated Output Files after running the Code-

From `train.py`

File	Type	Description
`model.pkl`	Binary	The trained Decision Tree model (~65 KB)
`symptoms.pkl`	Binary	List of all 134 symptom names
`disease_description.pkl`	Binary	Dictionary mapping each disease to its description
`disease_precaution.pkl`	Binary	Dictionary mapping each disease to its precautions
`symptom_severity.pkl`	Binary	Dictionary mapping each symptom to its severity score
`confusion_matrix.png`	Image	Heatmap showing predicted vs actual diseases
`classification_report.png`	Image	Bar chart of precision/recall/F1 per disease

Key Features-

Fuzzy Symptom Matching-
This was probably the feature I spent the most time on. It uses the `thefuzz` library with `token_sort_ratio` scoring. So if you type `headach` instead of `headache`, it'll still match. If you type `pain joint` instead of `joint_pain`, it figures that out too. And if the match is ambiguous, it shows you the top 3 options and lets you pick.

Symptom Severity Rating-
Every symptom has a severity score from 1 to 7 (from the dataset). The system calculates your average severity and rates it:
- **Low** (below 3)
- **Medium** (3 to 4.9)
- **High** (5 and above)

It also shows a nice severity table with bar indicators so you can see at a glance how serious each symptom is.

Confidence-Scored Predictions-
Instead of just giving one answer, the system uses `predict_proba()` to rank diseases by probability. It shows up to 3 predictions with ASCII confidence bars so you can see how confident it is. Only predictions with at least 1% confidence are shown.

Enriched Output-
- A description of the disease
- Up to 4 precautionary measures you can take
- Color-coded severity indicators

Automated Visual Evaluation-
- A confusion matrix heatmap with annotated counts
- A per-disease bar chart comparing precision, recall, and F1-score

These are helpful to quickly see how the model is doing across all diseases.

Project Structure

```
AI_disease_predection/
|
+-- train.py                    # Training script (186 lines)
+-- predict.py                  # Prediction script (166 lines)
+-- archive.zip                 # Source dataset
+-- README.md                   # Project documentation
+-- Project_Report.md           # This report
|
+-- model.pkl                   # Serialized trained model
+-- symptoms.pkl                # Symptom feature list
+-- disease_description.pkl     # Disease descriptions
+-- disease_precaution.pkl      # Disease precautions
+-- symptom_severity.pkl        # Symptom severity weights
|
+-- confusion_matrix.png        # Confusion matrix visualization
+-- classification_report.png   # Per-disease metrics chart
```

Sample Output-

Training-

```
Found ZIP: archive.zip
 Using pre-split files: Training.csv + Testing.csv

[+] Model Accuracy: 97.62%

[+] Generating confusion matrix...
```

```
[+] Saved: confusion_matrix.png
[+] Generating per-disease accuracy chart...
[+] Saved: classification_report.png

============================================================
 CLASSIFICATION REPORT
============================================================
[... full classification report ...]
============================================================

 Saved descriptions for 42 diseases
 Saved precautions for 42 diseases
 Saved severity weights for 133 symptoms
 Cleaned up temp files
 Model trained and saved successfully!
```

Prediction (Sample Output)-

```
Available symptoms:
- abdominal_pain
- abnormal_menstruation
- acidity
- acute_liver_failure
- altered_sensorium
- anxiety
- back_pain
- belly_pain
- blackheads
- bladder_discomfort
- blister
- blood_in_sputum
- bloody_stool
- blurred_and_distorted_vision
- breathlessness
- brittle_nails
- bruising
- burning_micturition
- chest_pain
- chills
- cold_hands_and_feet
- coma
- congestion
- constipation
- continuous_feel_of_urine
- continuous_sneezing
- cough
- cramps
- dark_urine
- dehydration
- depression
- diarrhoea
- dyschromic_patches
- distention_of_abdomen
- dizziness
- drying_and_tingling_lips
- enlarged_thyroid
- excessive_hunger
- extra_marital_contacts
- family_history
```

```
ankeshraj@ANKESHs-MacBook-Air ai ml project % trap 'rm /var/folders/5p/bpyhryj53q155_h1q5hcb8580000gn/T/antigravi
ty-sandbox-267280708.sb' EXIT; sandbox-exec -f /var/folders/5p/bpyhryj53q155_h1q5hcb8580000gn/T/antigravity-sandb
ox-267280708.sb /bin/bash -c 'echo "itching, skin rash, nodal skin eruptions" | python3 predict.py 2>/dev/null |
tail -30'
  Skin Rash                                3/7
  Nodal Skin Eruptions                     4/7

TOP PREDICTIONS

Fungal infection
[                                        ] 100.0%

Description:
 In humans, fungal infections occur when an
invading fungus takes over an area of the body
and is too much for the immune system to handle.
Fungi can live in the air, soil, water, and
plants. Some fungi live naturally in the human
body. Like many microbes, there are helpful fungi
and harmful fungi.

Precautions:
1. Bath twice
2. Use detol or neem in bathing water
3. Keep the infected area dry
4. Use clean cloths

This tool is for educational purposes only.
Always consult a qualified doctor for medical advice.

ankeshraj@ANKESHs-MacBook-Air ai ml project %
```

Limitations-

1. **Small dataset**- Only 306 training samples — that is not nearly enough for anything clinical
2. **Static symptoms**- The dataset is fixed, so it doesn't know about new diseases or symptoms
3. **Single algorithm**- I only used a Decision Tree — using something like Random Forest or XGBoost would probably give better results
4. **Label quality**- There is a typo in the dataset (`brain hemorrhageH`) that messes up one disease's classification
5. **No clinical validation**- This has not been tested against real patient data
6. **Binary features only**- It does not capture how intense a symptom is or how long you've had it

Future Enhancements-

- **Web Interface**- Build a Flask or React frontend so people do not have to use the terminal
- **REST API**- Add a `/predict` endpoint so other apps could use the model
- **Multiple Models**- Try Random Forest, SVM, Boost, and maybe even a neural network to compare
- **Bigger Dataset**- Use real-world clinical data for better accuracy
- **Symptom Autocomplete**- Live suggestions as you type so you don't even need fuzzy matching
- **Multi-Language Support**- Let people type symptoms in their own language
- **Prediction History**- Save all predictions to a CSV or database for later analysis
- **Patient Profiling**- Factor in age and gender for more accurate predictions

Conclusion-

The system hits **97.62% accuracy** on the test data, and **41 out of 42 diseases** get classified perfectly. The one that does not is because of a typo in the dataset, not because the model is bad.

The fuzzy matching system turned out to be useful — it makes the tool way more forgiving when you are typing symptoms. And the enriched output (descriptions, precautions, severity ratings) makes it informative rather than just giving you a disease name and leaving you hanging.

This is just an educational project and you should not use it to diagnose yourself. But it does show that even a simple ML model like a Decision Tree can do a solid job at mapping symptoms to diseases when the data is clean and how we can use it future. If I were to take this further, I'd want to use a larger dataset, try out multiple algorithms, and build a proper web interface.

References-

1. Scikit-learn Documentation — Decision Tree Classifier
https://scikit-learn.org/stable/modules/tree.html

2. Kaggle — Disease Symptom Description Dataset
https://www.kaggle.com/datasets/itachi9604/disease-symptom-description-dataset

3. TheFuzz Library — Fuzzy String Matching
https://github.com/seatgeek/thefuzz

4. Matplotlib Documentation
https://matplotlib.org/stable/contents.html

5. Pandas Documentation
https://pandas.pydata.org/docs/

6. VITyarthi Course

Prepared by: Ankesh Raj
Date: March 29, 2026
Project: AI Disease Prediction System